

Data Structures and Algorithms

iactavis

I. Why Study DS&A?

Problem: We have a lot of data. We have a finite amount of computing power, time and memory. We require fairly complicated things from our raw datasets. The question arises—how do we store data?

Solution: we develop data structures and associated algorithms to suit our needs.

II. Asymptotic Complexity

We would like to compare of two algorithms, which grows faster? We would like to compare the scalability of two algorithms. We therefore define an upper-bound, lower-bound and both.

Definition 1

Let f and g be functions from $\mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0}$. We say that $f(x) = \mathcal{O}(g(x))$ if there exist positive x_0 and c such that for all $x > x_0$, $f(x) \leq cg(x)$. Another way of saying this is $\frac{f}{g}$ is eventually bounded.

An alternative convention is to say $\mathcal{O}(g(x))$ is the set of all functions $\hat{f}$ such that $\hat{f} = \mathcal{O}(g(x))$. So we may instead say $f \in \mathcal{O}(g(x))$.

Definition 2

Let f and g be functions from $\mathbb{R}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0}$. We say that $f(x) = \Omega(g(x))$ if there exist positive x_0 and c such that for all $x > x_0$, $cg(x) \leq f$. g will eventually become a lower bound of f .

As before, $f \in \Omega(g(x))$.

Definition 3

If $f = \mathcal{O}(g)$ and $f = \Omega(g)$, we say that $f = \Theta(g)$.

Definition 4

Complexity	Name
$\mathcal{O}(\log n)$	logarithmic
$\mathcal{O}(n)$	linear
$\mathcal{O}(n^2)$	quadratic
$\mathcal{O}(n^3)$	cubic
$\mathcal{O}(n^{\text{constant}})$	polynomial
$\mathcal{O}(e^n)$	exponential
$\mathcal{O}(n!)$	factorial

II. i. Constant Factors

The $\Theta(f)$ system only cares about asymptotic time complexity, but not about actual running time.

To give an example, consider $f(x) = 10^{100}x$ and $g(x) = x$. Then $f = \Theta(g)$ but the running time of

f is impractical. So even if an algorithm is better asymptotically, it need not be better in practical use. It only guarantees that it will be better for large datasets. We will see examples of this later.

III. Basic Data Structures

IV. Sorting

Problem

Given a sequence of numbers $A = (a_i)_{i=1}^n$ and a *total ordering* on A , denoted by $\leq$, return a permutation of A , say A' , that satisfies $a'_1 \leq a'_2 \leq \dots \leq a'_n$.

For example, consider the sequence $(26, 58, 31, 41, 67, 41)$ and the order to be the ascending order. Then the expected output is $(26, 31, 41, 41, 58, 67)$.

A *total ordering* $\leq$ satisfies

1. $a \leq a$ (reflexive)
2. if $a \leq b$ and $b \leq c$ then $a \leq c$ (transitive)
3. if $a \leq b$ and $b \leq a$ then $a = b$ (antisymmetric)
4. $a \leq b$ or $b \leq a$ (totality)

We will usually use numeric or lexicographic sorting.